Machine Learning for Business

GTSC2143

Lecture 11 Recommendation System

Instructor: Dr. Lily Bei Luo

AY2025-2026 Semester 1

北京师范大学 香港浸会大学 联合国际学院
BEIJING NORMAL UNIVERSITY · HONG KONG BAPTIST UNIVERSITY
UNITED INTERNATIONAL COLLEGE

Where we see recommender systems

Recommender systems

Personalization
is transforming our experience of the world

Information overload

Browsing is "history"

- Need new ways to discover content

YouTube

100 Hours a Minute

What do I care about?

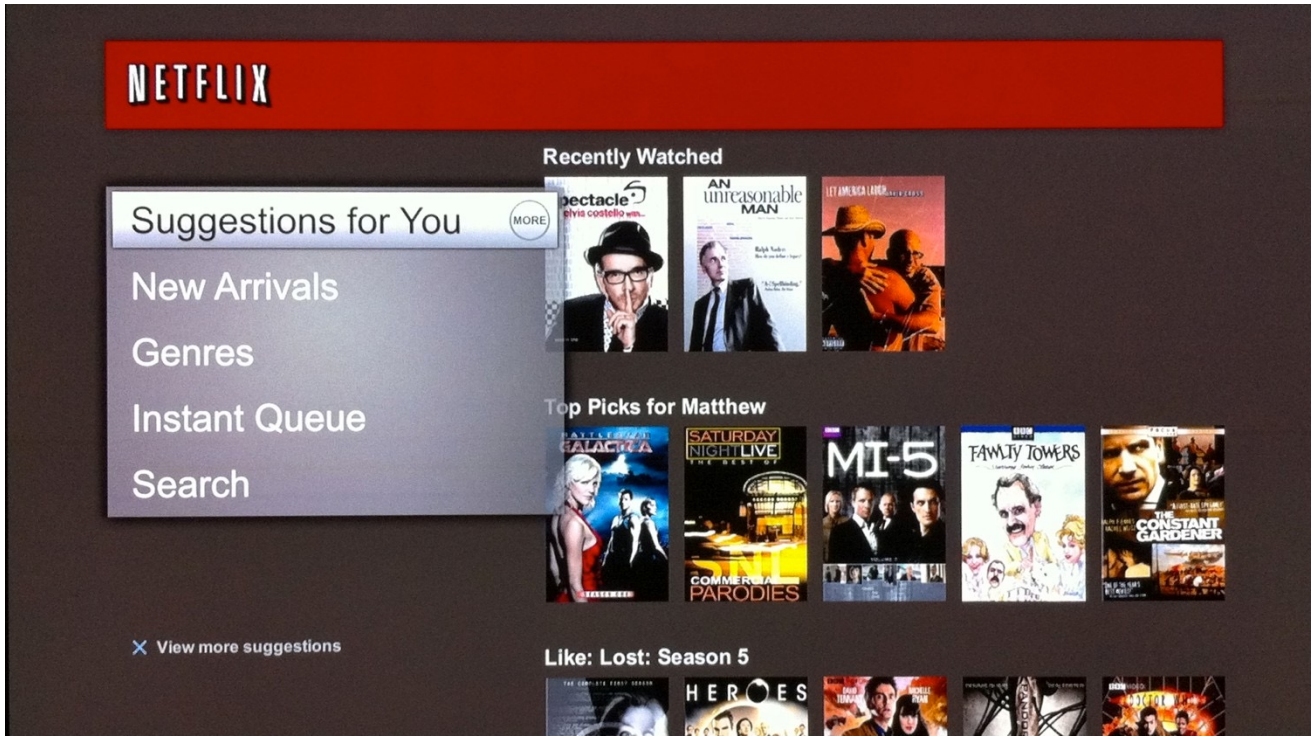
Personalization: Connects *users & items*

viewers

videos

Recommender systems

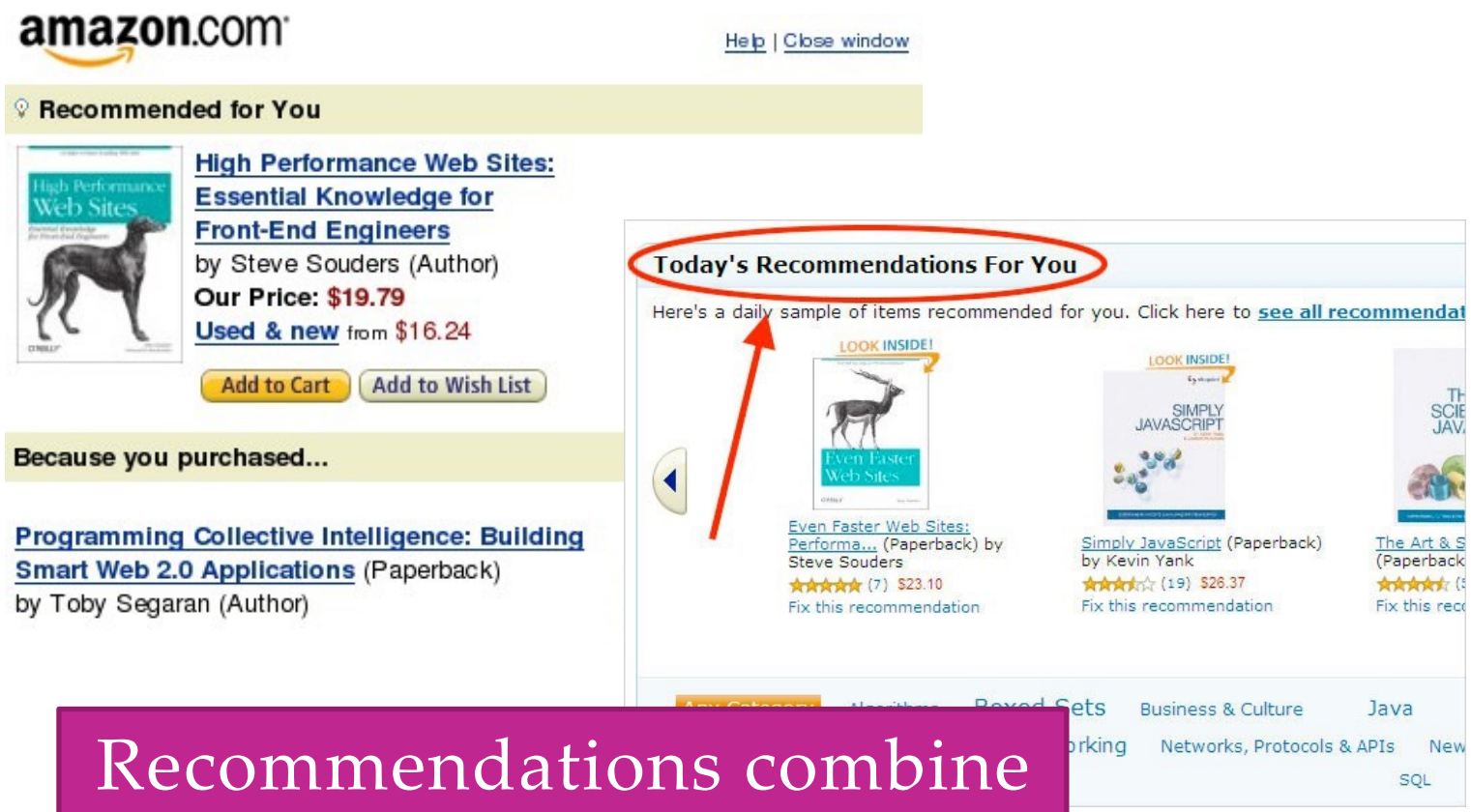
Movie recommendations



Connect users with movies they may want to watch

Recommender systems

Product recommendations



Recommendations combine global & session interests

Recommender systems

Music recommendations

Recommendations form coherent & diverse sequence

Recommender systems

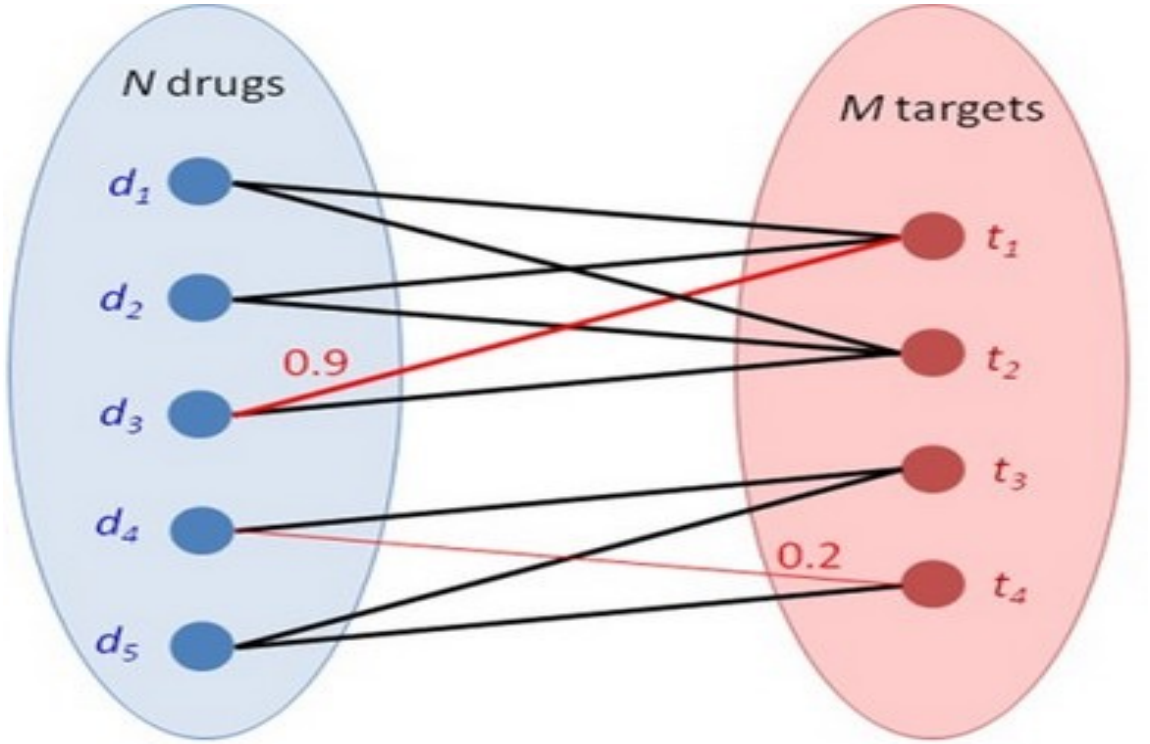
Friend recommendations

Users and “items”

Recommender systems

Drug-target interactions

Cobanoglu et al. '13



What drugs should we "repurpose" for some disease?

Building a recommender system

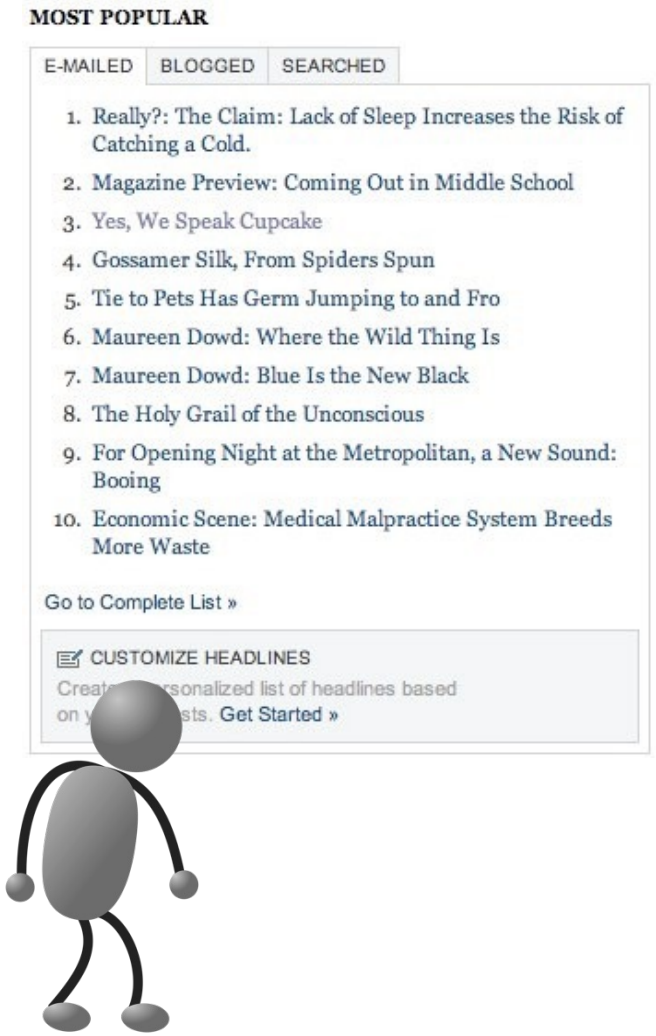
Solution 0: Popularity

Solution 0

Popularity

- What are people viewing now?
  - Rank by global popularity

- Limitation:
  - No personalization



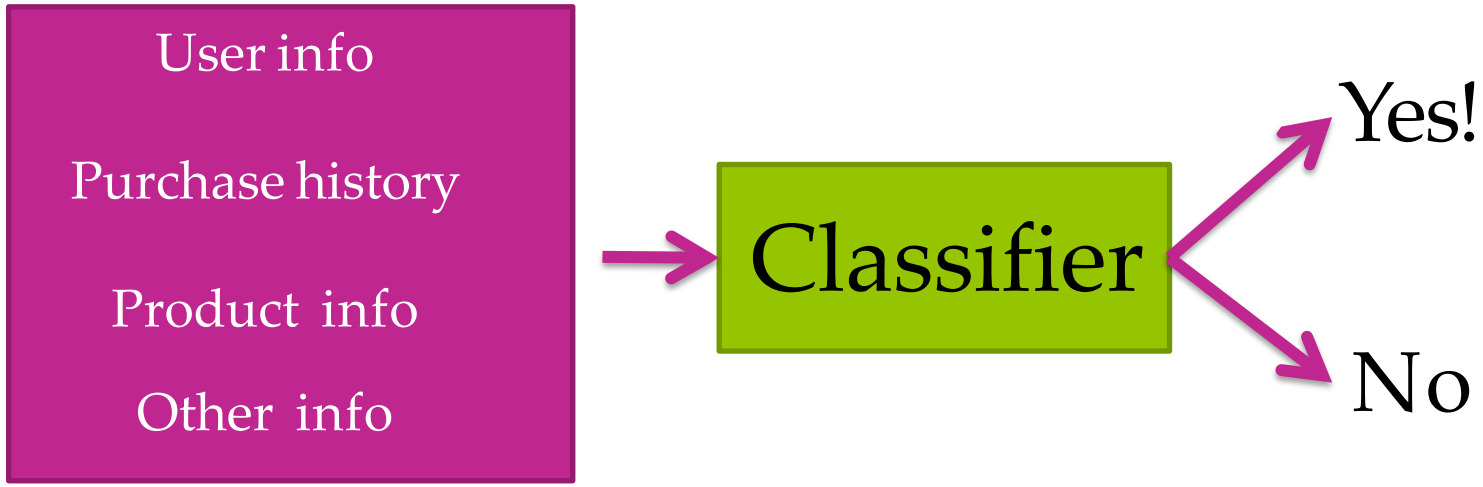
Simplest approach

Solution 1: Classification Model

Solution 1

Classification model

- What's the probability I'll buy this product?

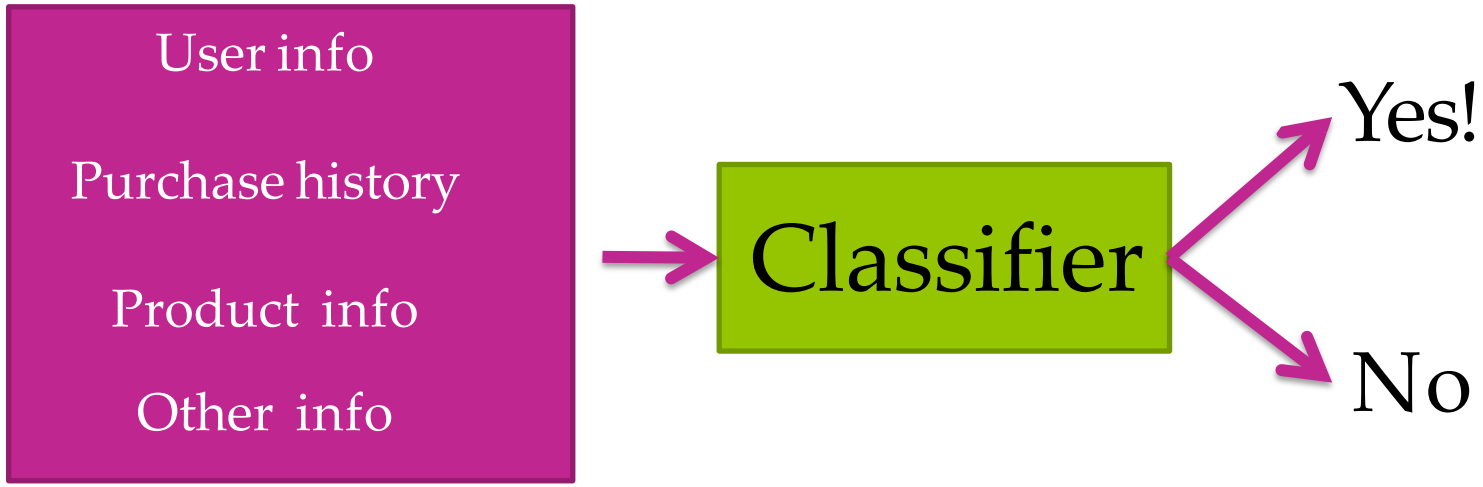


- Pros:
  - Personalized: Considers user info & purchase history
  - Features can capture context: Time of the day, what I just saw,…
  - Even handles limited user history: Age of user, …

Solution 1

Classification model

Limitations of classification approach



- Features may not be available
- Often doesn't perform as well as collaborative filtering methods (next)

Solution 2: People who bought this also bought…

Solution 2

Co-occurrence matrix

- People who bought *diapers* also bought *baby wipes*

- Matrix C:
  store # users who bought both items *i & j*
  - (# items x # items) matrix

	beer	**diapers**	**baby wipes**	**milk**	**…**
beer	0	7	7	2	
diapers	7	0	9	4	
baby wipes	7	9	0	1	
milk	2	4	1	0	
…					

- Symmetric: # purchasing *i & j* same as # for *j & i* ($C_{ij} = C_{ji}$)

Solution 2

Making recommendations using co-occurences

- User  purchased *diapers*

1. Look at *diapers* row of matrix

	beer	**diapers**	**baby wipes**	**milk**	**…**
beer	0	7	7	2	
diapers	**7**	**0**	**9**	**4**	
baby wipes	7	9	0	1	
milk	2	4	1	0	
…					

2. Recommend other items with largest counts
   - *baby wipes*, *milk*, *baby food*,…

Solution 2

Co-occurrence matrix must be normalized

- What if there are very popular items?
  - Popular baby item: *Pampers Swaddlers diapers*

  - For any baby item (e.g., *i*=*Sophie giraffe* ) large count C_{ij} for *j*=*Pampers Swaddlers*

- Result:
  - Drowns out other effects
  - Recommend based on popularity

These co-occurrences happen not because A or C is strongly related to the popular item B, but simply because "B is so popular that everyone buys it."

Solution 2

Normalize co-occurrences: Similarity matrix

- Jaccard similarity: normalizes by popularity
  - Who purchased *i* **and** *j* *divided by* who purchased *i* **or** *j*

From simple counts to similarity measures

	beer	**diapers**	**baby wipes**	**milk**
beer	0	7/16	7/16	2/16
diapers	7/20	0	9/20	4/20
baby wipes	7/17	9/17	0	1/17
milk	2/7	4/7	1/7	0

- Many other similarity metrics possible, e.g., cosine similarity

Solution 2

Limitations

- Only current page matters, no history
  - Recommend similar items to the one you bought

- What if you purchased many items?
  - Want recommendations based on purchase history

Solution 2

(Weighted) Average of purchased items

- User  bought items {*diapers*, *milk*}
  - Compute user-specific score for each item *j* in inventory by combining similarities:

$$Score(\ , baby\ wipes) = \tfrac{1}{2}\,(S_{baby\ wipes,\ diapers} + S_{baby\ wipes,\ milk})$$

  - Could also weight recent purchases more

- Sort *Score*(, *j*) and find item *j* with highest similarity

Solution 2

Limitations

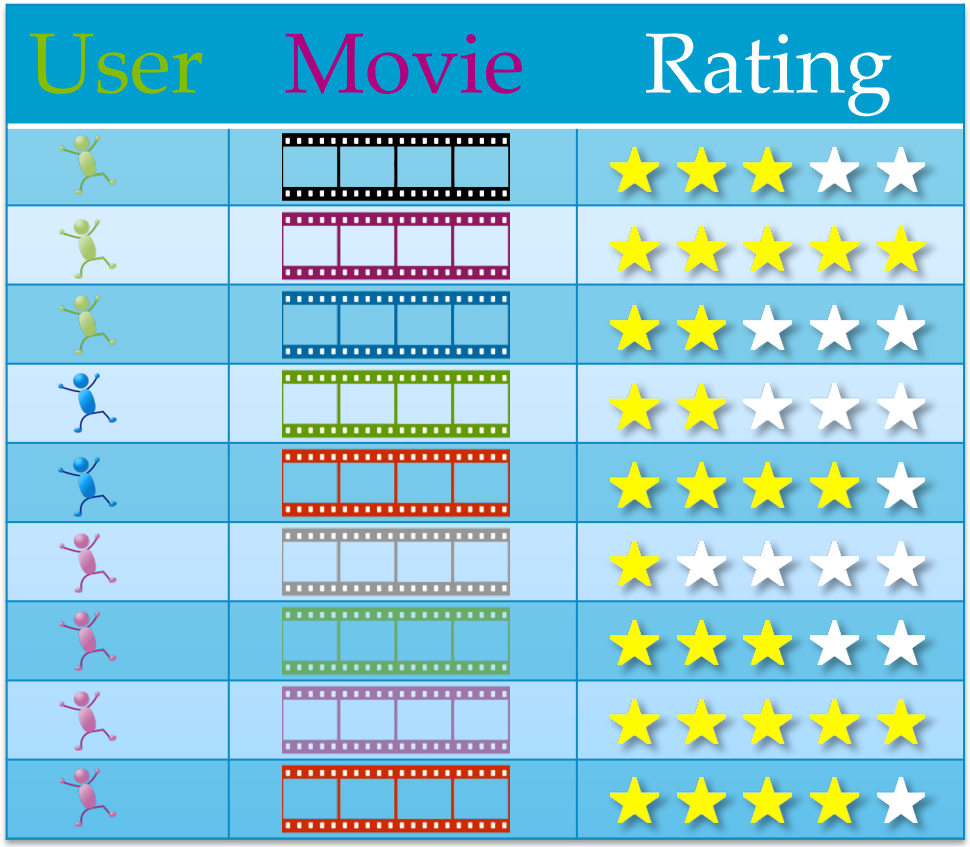
- Does not utilize:
  - context (e.g., time of day)
  - user features (e.g., age)
  - product features (e.g., baby vs. electronics)
- Cold start problem
  - What if a new user or product arrives?

Solution 3: Discovering hidden structure by matrix factorization

Solution 3

Movie recommendation

- Users watch movies and rate them

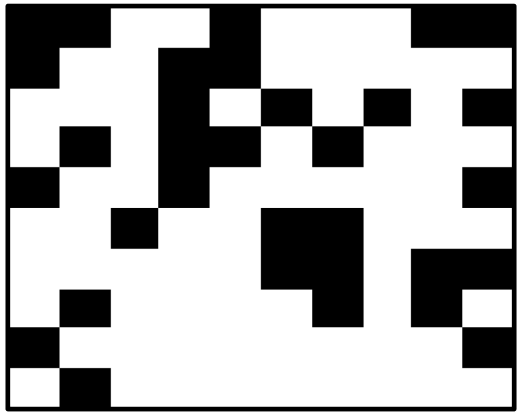


Each user only watches a few of the available movies

Solution 3

Matrix completion problem

Rating=



- Data: Users score some movies

  Rating(*u*,*v*) known for black cells
  Rating(*u*,*v*) unknown for white cells

- Goal: Filling missing data?

Solution 3

Suppose we had *d* topics for each user and movie

- Describe movie *v*  with topics R_v
  - How much is it action, romance, drama,…

$R_v = [\,0.3 \quad 0.01 \quad 1.5 \quad \ldots\,]$

- Describe user *u*  with topics L_u
  - How much she likes action, romance, drama,…

$L_u = [\,2.5 \quad 0 \quad 0.8 \quad \ldots\,]$

- *Rating(u,v)* is the product of the two vectors

$R_v = [\,0.3 \quad 0.01 \quad 1.5 \quad \ldots\,] \rightarrow 0.3*2.5 + 0 + 1.5*0.8 + \ldots = 7.2$

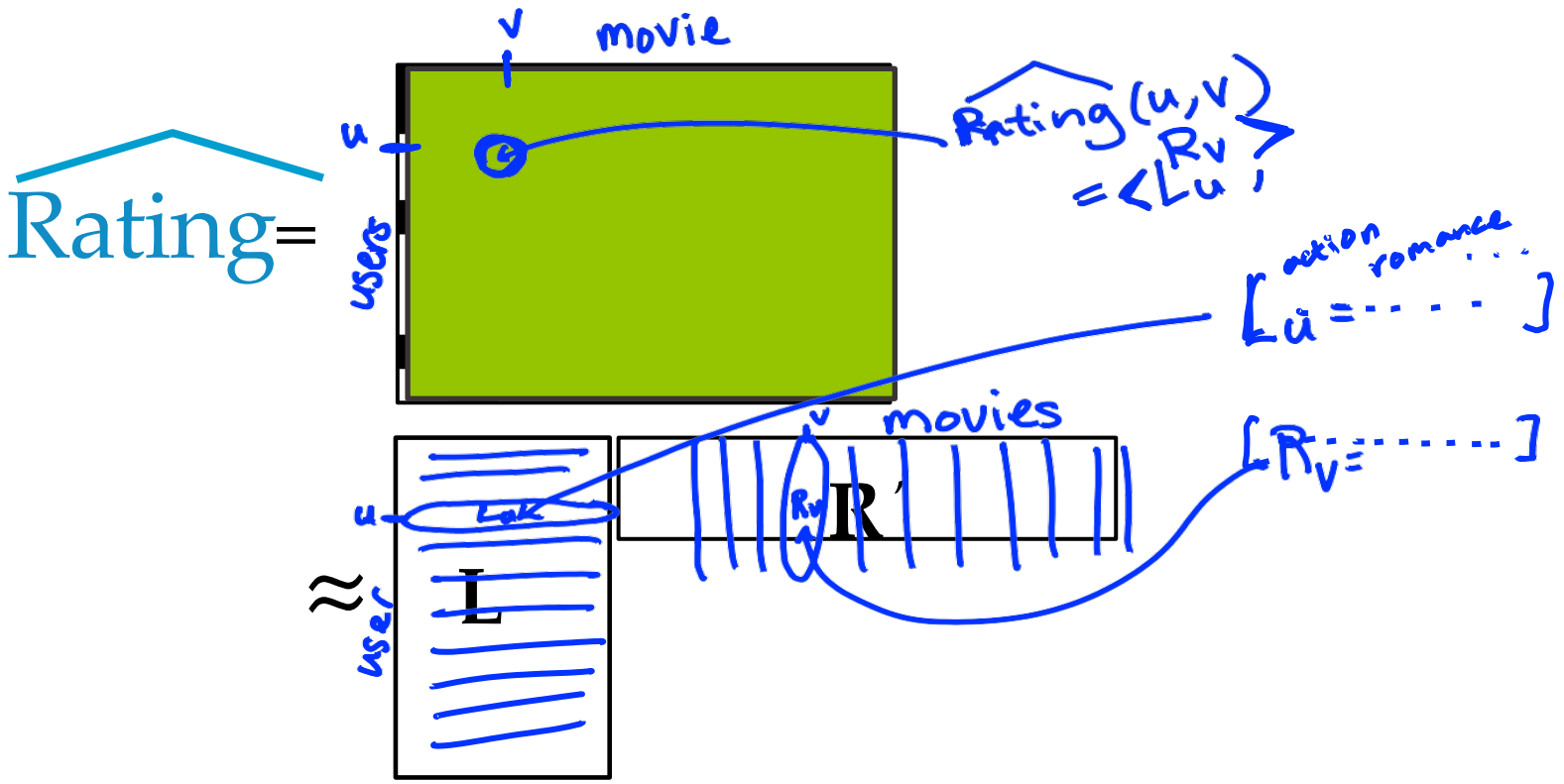
$L_u = [\,2.5 \quad 0 \quad 0.8 \quad \ldots\,]$

$L_{u'} = [\,0 \quad 3.5 \quad 0.01 \quad \ldots\,] \rightarrow 0 + 0.01*3.5 + 1.5*0.04\ldots = 0.8$

- Recommendations: sort movies user hasn't watched by *Rating(u,v)*

Solution 3

Predictions in matrix form

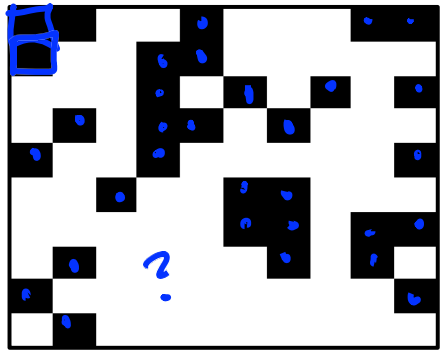


But we don't know topics of users and movies…

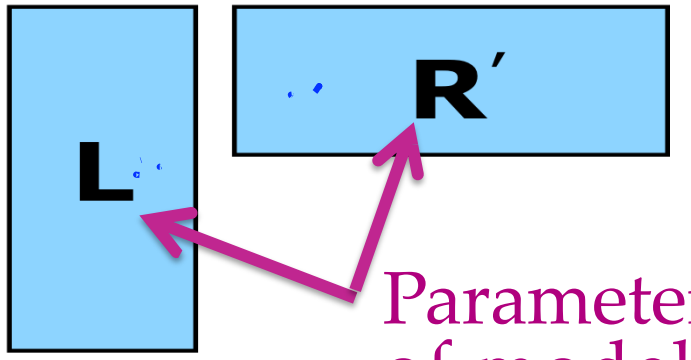
Solution 3

Matrix factorization model: Discovering topics from data

Rating=



≈



Parameters of model

$RSS(L,R) = \sum_{(u,v)} \left(\text{Rating}(u,v) - \langle L_u, R_v \rangle\right)^2$ ← predicted rating

+ [include all (u',v') pairs where Rating(u',v') are available]

Many efficient algorithms for

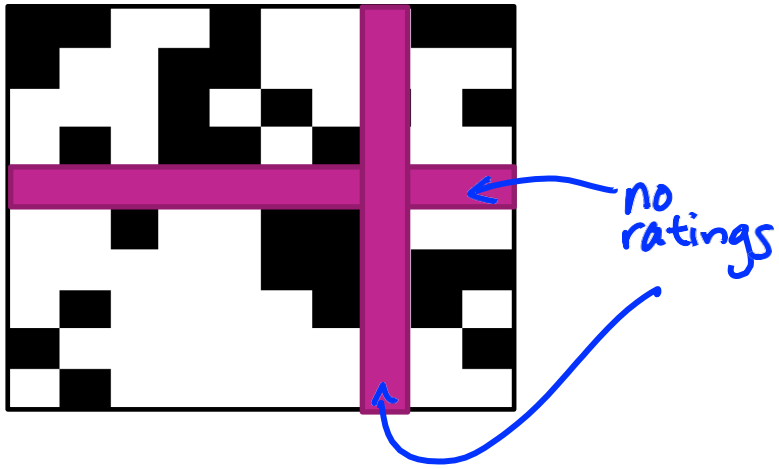
- Only use observed values to estimate "topic" vectors $\hat{L}_u$ and $\hat{R}_v$
- Use estimated $\hat{L}_u$ and $\hat{R}_v$ for recommendations

Solution 3

Limitations of matrix factorization

- Cold-start problem
  - This model still cannot handle a new user or movie

Rating=



Bringing it all together: Featurized matrix factorization

Solution 4

Combining features and discovered topics

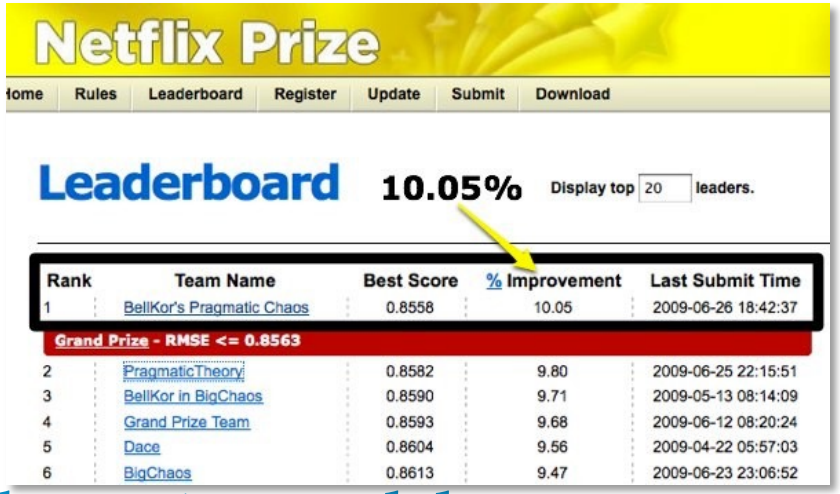
- Features capture context
  - *Time of day, what I just saw, user info, past purchases,…*
- Discovered topics from matrix factorization capture groups of users who behave similarly
  - *Women from Seattle who teach and have a baby*

- Combine to mitigate cold-start problem
  - Ratings for a new user from features only
  - As more information about user is discovered, matrix factorization topics become more relevant

Solution 4

Blending models

- Squeezing last bit of accuracy by blending models
- Netflix Prize 2006-2009
  - 100M ratings
  - 17,770 movies
  - 480,189 users
  - Predict 3 million ratings to highest accuracy
  - **Winning team blended over 100 models**



A performance metric for recommender systems

Performance metric

The world of all baby products

Performance metric

User likes subset of items

Question: How to measure this performance?

Performance metric

Why not use classification accuracy?

- Classification accuracy = fraction of items correctly classified (*liked* vs. *not liked*)

- Here, not interested in what a person *does not like*

- Rather, how quickly can we discover the relatively few *liked* items?
  - (Partially) an imbalanced class problem

Performance metric

How many liked items were recommended?

Recall

$$\frac{\#\ \text{liked \& shown}}{\#\ \text{liked}}$$

$= \frac{3}{5}$

Performance metric

How many recommended items were liked?

Precision

$$\frac{\#\ \text{liked \& shown}}{\#\ \text{shown}}$$

$$= \frac{3}{11}$$

Performance metric

Maximize recall: Recommend everything

Recall

liked & shown
liked

Performance metric

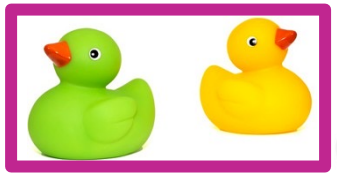
Resulting precision?

Precision

$$\frac{\#\ \text{liked \& shown}}{\#\ \text{shown}}$$

Performance metric

Optimal recommender



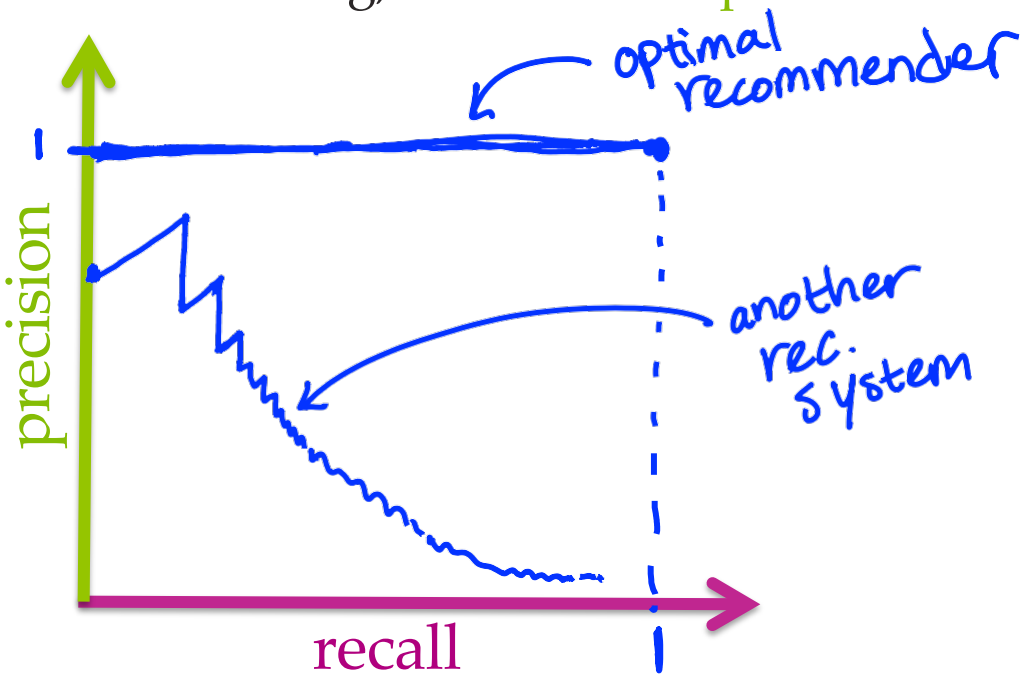
Recall =1
Precision = 1

Performance metric

Precision-recall curve

- Input: A specific recommender system
- Output: Algorithm-specific precision-recall curve

- To draw curve, vary threshold on # items recommended
  - For each setting, calculate the precision and recall



Overview of Recommender Methods

Overview of recommender methods

Three families

1. **Content-based methods**
   - Use item attributes (text, tags, feature vectors) and match them to items the user liked in the past.

➢ **Advantages:**
   - Does not require data from other users

➢ **Disadvantages:**
   - Recommendations can become narrow
   - Depends on high-quality features

In Solution 1, we use user history to predict the probability of buying a specific product.

Overview of recommender methods

Three families

2. Collaborative Filtering

- User-based CF
- Item-based CF
  - co-occurrence matrix
  - item-item similarity
- Latent factor models (Matrix Factorization)

➢ **Advantages:**

- Does not require item features

➢ **Disadvantages:**

- cold start and sparsity issues are significant

- In Solution 2, we introduce **item-based CF**, which recommends items similar to those the user has interacted with.
- **User-based CF** follows the same principle but looks for **users with similar behavior and recommends items** they liked.

Overview of recommender methods

Three families

3. Hybrid Models

- Combine content features + collaborative filtering
- Feature-aware matrix factorization
- Model blending / ensembles

Strong performance in large-scale industrial systems

Evaluation — Beyond Classification Accuracy

Key metrics:

- Precision: fraction of recommended items that users like
- Recall: fraction of liked items that are retrieved
- Precision–Recall curve and AUC

Note:

1. **Recommendation is not a standard classification task.**
2. **The goal is not to classify every item correctly but to surface the few relevant items as efficiently as possible.**

Python Implementation

Python implementation

Co-occurrence Matrix

```
import numpy as np
import pandas as pd
# generate sample data
data = {
    "user": [
        1,1,1,       # Milk, Bread, Eggs
        2,2,         # Bread, Butter
        3,3,3,       # Milk, Butter, Banana
        4,4,4,       # Milk, Bread, Butter
        5,5,         # Eggs, Banana
        6,6,6,       # Bread, Milk, Banana
        7,7          # Milk, Butter
    ],
    "item": [
        "Milk","Bread","Eggs",
        "Bread","Butter",
        "Milk","Butter","Banana",
        "Milk","Bread","Butter",
        "Eggs","Banana",
        "Bread","Milk","Banana",
        "Milk","Butter"
    ]
}
df = pd.DataFrame(data)
```

```
# item co-occurrence matrix
items = df["item"].unique()
co_matrix = pd.DataFrame(0, index=items, columns=items)

for user, group in df.groupby("user"):
    purchased = list(group["item"])
    for i in purchased:
        for j in purchased:
            if i != j:
                co_matrix.loc[i, j] += 1

print("Co-occurrence matrix:")
print(co_matrix)
```

```
Co-occurrence matrix:
        Milk  Bread  Eggs  Butter  Banana
Milk       0      3     1       3       2
Bread      3      0     1       2       1
Eggs       1      1     0       0       1
Butter     3      2     0       0       1
Banana     2      1     1       1       0
```

Python implementation

Item-based CF (cosine similarity)

```
from sklearn.metrics.pairwise import cosine_similarity

# convert co-occurrence matrix to numpy
M = co_matrix.values
sim = cosine_similarity(M)

# similarity dataframe
item_sim = pd.DataFrame(sim, index=items, columns=items)
print("Item similarity matrix:")
print(item_sim.round(3))

# simple recommendation: items similar to a given item
def recommend_similar(item, top_k=3):
    scores = item_sim[item].sort_values(ascending=False)
    return scores.iloc[1:top_k+1]   # skip itself

print("\nItems similar to 'Milk':")
print(recommend_similar("Milk"))
```

```
Item similarity matrix:
         Milk  Bread   Eggs  Butter  Banana
Milk    1.000  0.485  0.602   0.446   0.552
Bread   0.485  1.000  0.596   0.690   0.878
Eggs    0.602  0.596  1.000   0.926   0.655
Butter  0.446  0.690  0.926   1.000   0.808
Banana  0.552  0.878  0.655   0.808   1.000

Items similar to 'Milk':
Eggs      0.601929
Banana    0.551677
Bread     0.484544
Name: Milk, dtype: float64
```

Python implementation

Matrix factorization

```
# create user-item matrix: buy=1

users = df["user"].unique()
items = df["item"].unique()
user_index = {u:i for i,u in enumerate(users)}
item_index = {it:i for i,it in enumerate(items)}

R = np.zeros((len(users), len(items)))

for _, row in df.iterrows():
    R[user_index[row["user"]], item_index[row["item"]]] = 1

print("User-Item Matrix (implicit feedback):")
print(pd.DataFrame(R, index=users, columns=items))
```

```
User-Item Matrix (implicit feedback):
   Milk  Bread  Eggs  Butter  Banana
1   1.0    1.0   1.0     0.0     0.0
2   0.0    1.0   0.0     1.0     0.0
3   1.0    0.0   0.0     1.0     1.0
4   1.0    1.0   0.0     1.0     0.0
5   0.0    0.0   1.0     0.0     1.0
6   1.0    1.0   0.0     0.0     1.0
7   1.0    0.0   0.0     1.0     0.0
```

Create user-item matrix to indicate buy or not

Python implementation

Matrix factorization

```
# --- Matrix Factorization (SGD) ---
num_users, num_items = R.shape
K = 2  # latent factors

P = np.random.rand(num_users, K)
Q = np.random.rand(num_items, K)

lr = 0.01
epochs = 2000

for _ in range(epochs):
    for u in range(num_users):
        for i in range(num_items):
            if R[u, i] > 0:  # only update known interactions
                pred = np.dot(P[u], Q[i])
                err = R[u, i] - pred

                P[u] += lr * err * Q[i]
                Q[i] += lr * err * P[u]

pred_matrix = np.dot(P, Q.T)

print("\nPredicted preference matrix:")
print(pd.DataFrame(pred_matrix.round(3), index=users, columns=items))
```

```
Predicted preference matrix:
    Milk  Bread   Eggs  Butter  Banana
1  1.004  0.998  0.998   1.004   1.006
2  1.001  0.999  0.997   1.001   1.002
3  0.999  0.975  0.988   0.998   1.002
4  1.000  1.000  0.996   1.000   1.001
5  0.999  1.016  1.001   1.001   0.999
6  0.998  1.004  0.996   0.999   0.999
7  0.999  1.004  0.998   1.000   1.001
```

Tutorial